IST 476: APPLIED INFORMATION SECURITY

Recognizing the Use of Steganography in Forensic Evidence

Bonus Lab Report

Prepared by: James O'Leary
Instructor: Dr. Joon Park
Institution: School of Information Studies, Syracuse University
Date: 11/11/2026

1. Executive Summary

This examination of the Beverly Gates evidence image tested if steganography was used to hide data. A hash search in Paraben's E3 matched a file named "temp17.exe.exe" (MD5 = FDEBDAC3B431DF68015DFC326B9A7042) to OpenPuff, a steganography tool. StegExpose then flagged a file named "lakeview.bmp" (MD5 = 14229A8A8BE6D6822B4D8F22B29E4842) in the exported "Received" folder as suspicious. StegExpose estimated 31,289 bytes of hidden data despite the fact the image looked normal in MS paint. The readme file in the same "Received" folder revealed a passphrase, "Lakeview". OpenPuff recovered a text file named "locations" that lists Chicago addresses with GPS coordinates.

2. Introduction

2.1 Purpose

This report documents a forensic examination of the Beverly Gates evidence case to determine whether steganography was used to conceal data. The examination identified steganography software on the evidence drive image, located an image file carrying hidden content, and recovered that content for analysis.

Scope: Hands-On Demonstrations, Parts 1 through 3, and Research Question 1 (IST 476). Applied Learning and Challenge and Analysis were not assigned.

2.2 Objectives

#	Objective
1	Detect known steganography software on an evidence drive image using file-hash searches.
2	Identify image files suspected of containing hidden data using StegExpose.
3	Extract concealed data with OpenPuff while preserving the original evidence.
4	Document the recovered contents and assess their relevance to the investigation.

2.3 Environment and Tools

All work was performed on vWorkstation (Windows Server 2019, 172.30.0.2) in the course cloud lab. Screenshots were captured with the lab's built-in screenshot tool.

Tool	Purpose
Paraben's E3	Hash-based search of the evidence image; file export
StegExpose	Statistical (LSB) steganalysis of exported images
Microsoft Paint	Visual inspection of the flagged image
OpenPuff	Extraction of the concealed file using the recovered passphrase

3. Part 1: Detecting Steganography Software on a Drive Image

3.1 Procedure

The Beverly Gates evidence case was opened in Paraben's E3 on vWorkstation. The "Hash Database1.pdh" was attached through the Hash Databases tab in E3. That database includes a group of known MD5 Hashes for Steganography tools. A Sorted Files Search compared the hashes of the case files against the database which returned one match.

3.2 Evidence

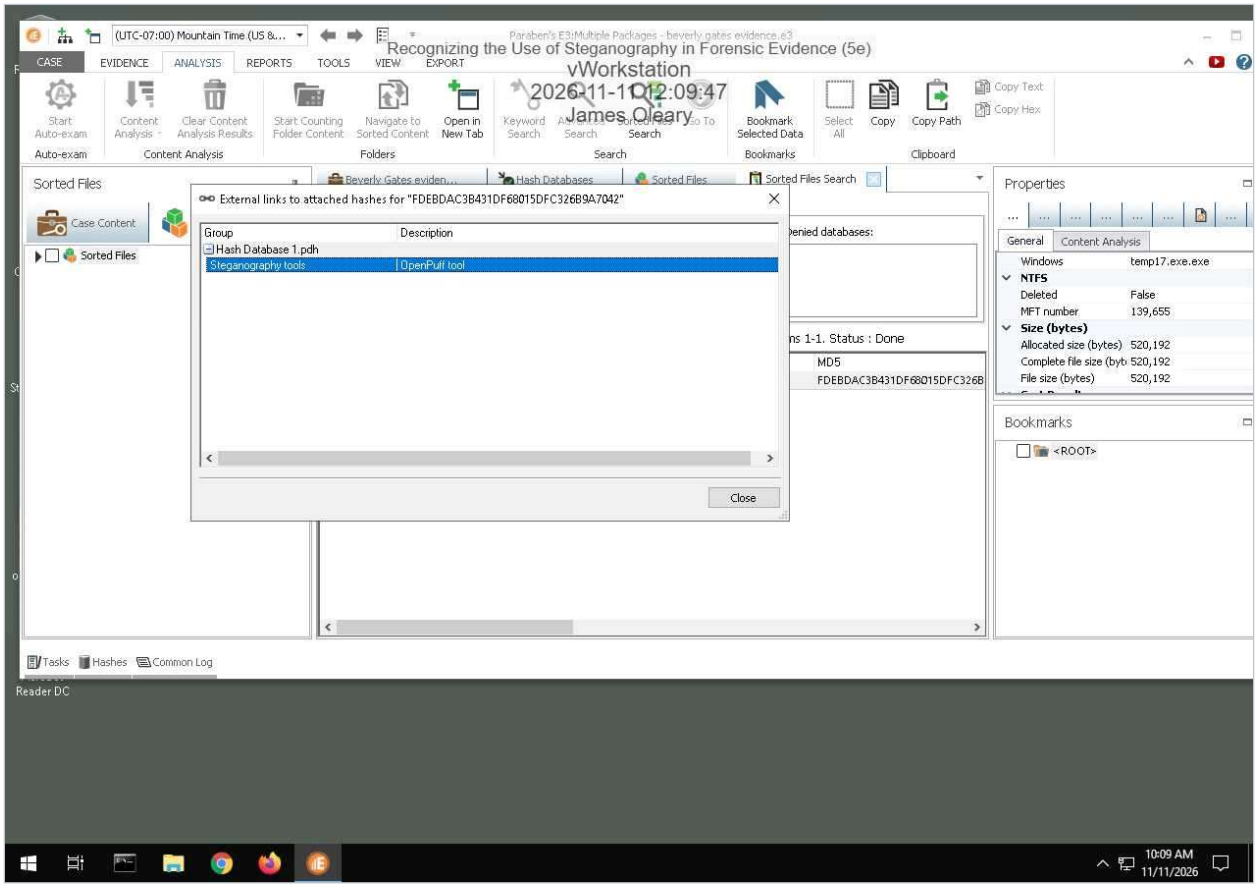


Figure 1. *Part 1, E3 hash search result and its description*

3.3 Analysis

The detected match was with OpenPuff, listed in the “Steganography tools” group of the hash database with a description listed as “OpenPuff tool”. On the evidence drive the file was stored as temp17.exe.exe (MD5 = FDEBDAC3B431DF68015DFC326B9A7042), a name meant to disguise the program as a temp file. A file name or extension can be changed easily, while a hash is calculated from the file’s contents. Renaming the file did nothing to hide it from the search because the file contents were identical. Of course, the limit here is that hash matching only catches 1 to 1 copies of known files.

4. Part 2: Detecting Hidden Data in Image Files

4.1 Procedure

The “Received” folder was exported from E3 which wrote a .MD5 hash file beside all exported images so they can be verified against the originals. StegExpose was then run against the whole “Received” folder. StegExpose flagged one file, “lakeview.bmp”, the file was opened in MS paint to see if there were any visible signs of tampering.

4.2 Evidence

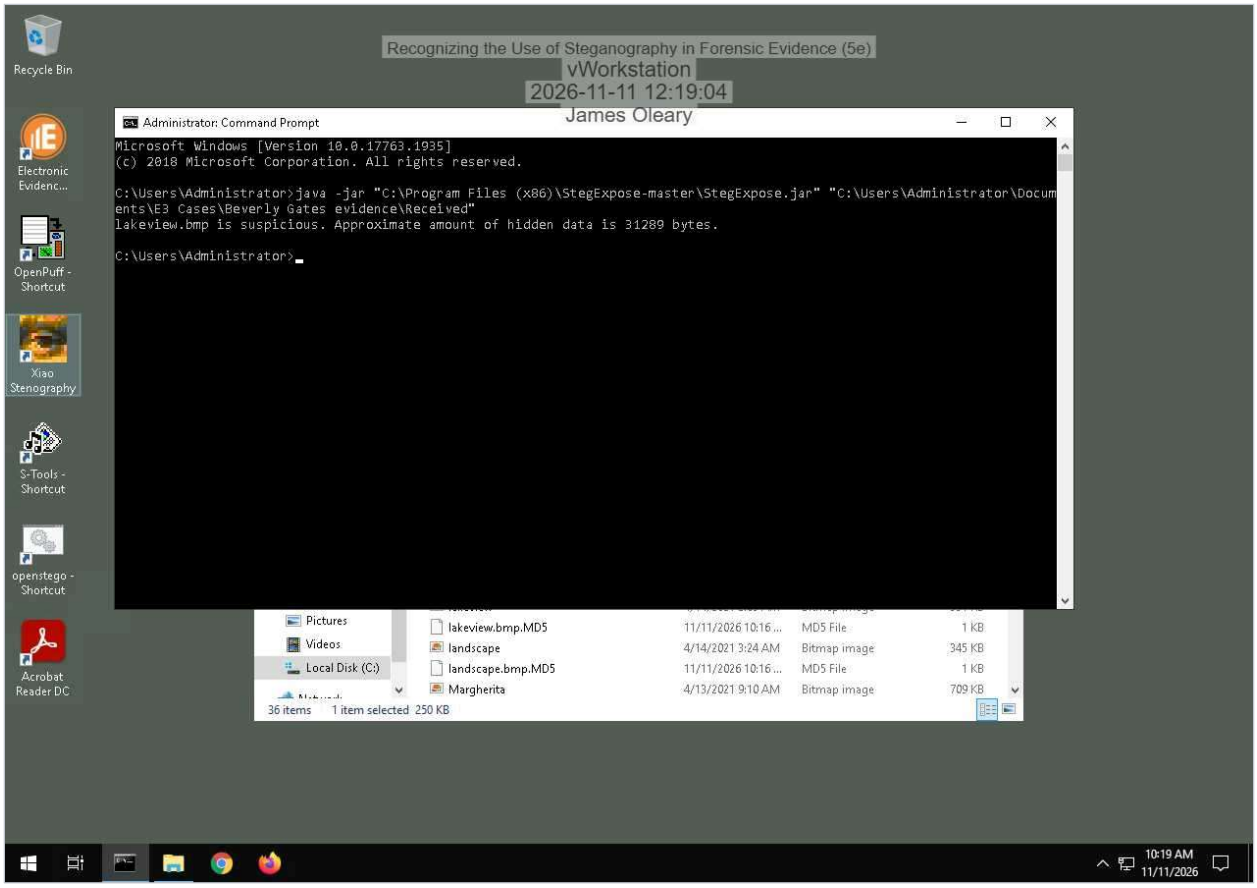


Figure 2. *Part 2, StegExpose scan results*

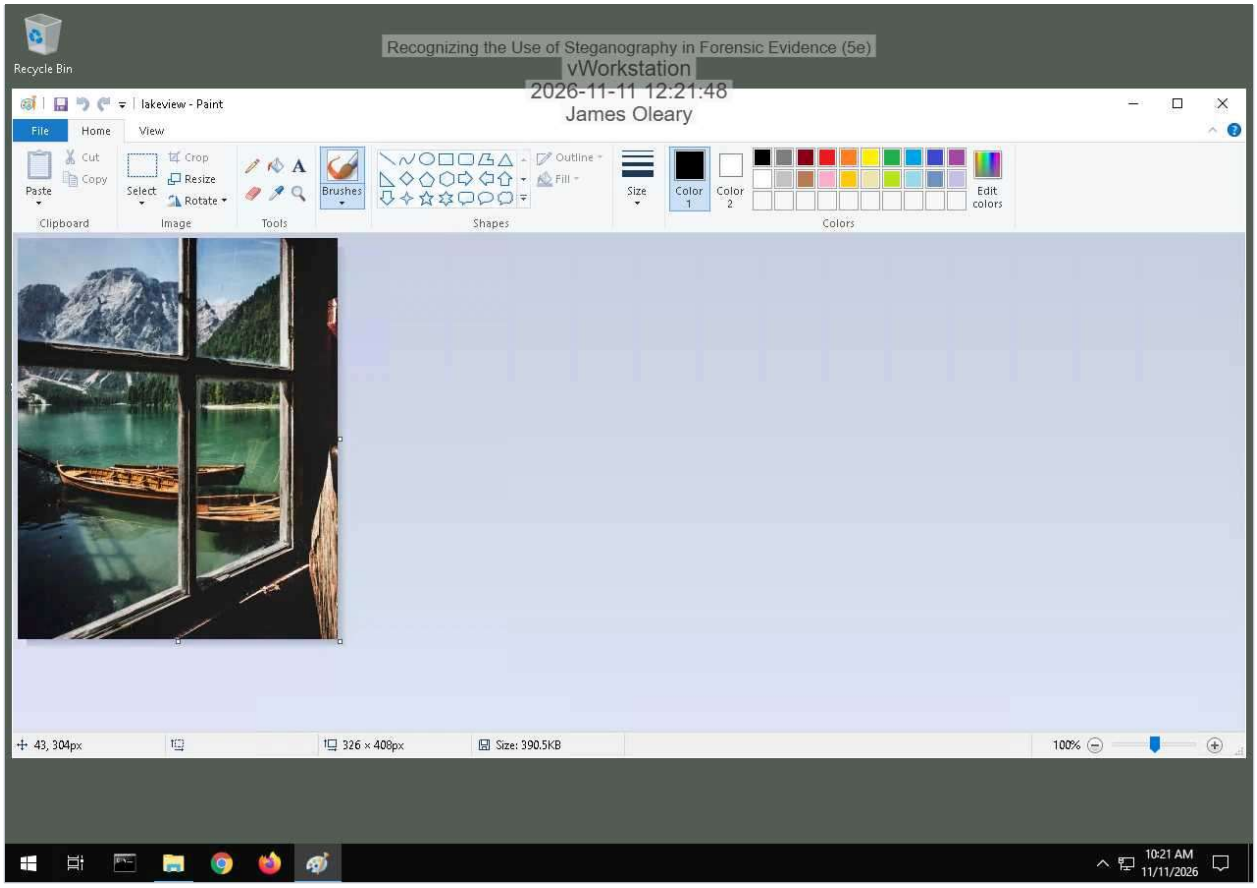


Figure 3. *Part 2, Suspicious image displayed in Microsoft Paint*

4.3 Analysis

StegExpose estimated “lakeview.bmp” contained 31289 bytes of hidden data. When opened in MS paint, the image (326 x 408 px, 390.5 kb) showed a normal photo. No noise, color banding, or distortion was observed. The file size lines up with the standard 24-bit bitmap so nothing was added to the end of the file. This fits the description for least significant bit embedding. This is why StegExpose was able to catch it while visual analysis could not.

5. Part 3: Extracting Hidden Data from Image Files

5.1 Procedure

The passphrase “Lakeview” was found in a Readme file in the exported “Received” folder, the same folder as the suspicious image. Using OpenPuff’s Unhide mode, the passphrase was entered and the exported “lakeview.bmp” was added as the carrier. Extraction ran and OpenPuff recovered a hidden file named “locations”. It was saved to desktop and opened in Notepad for review.

5.2 Evidence

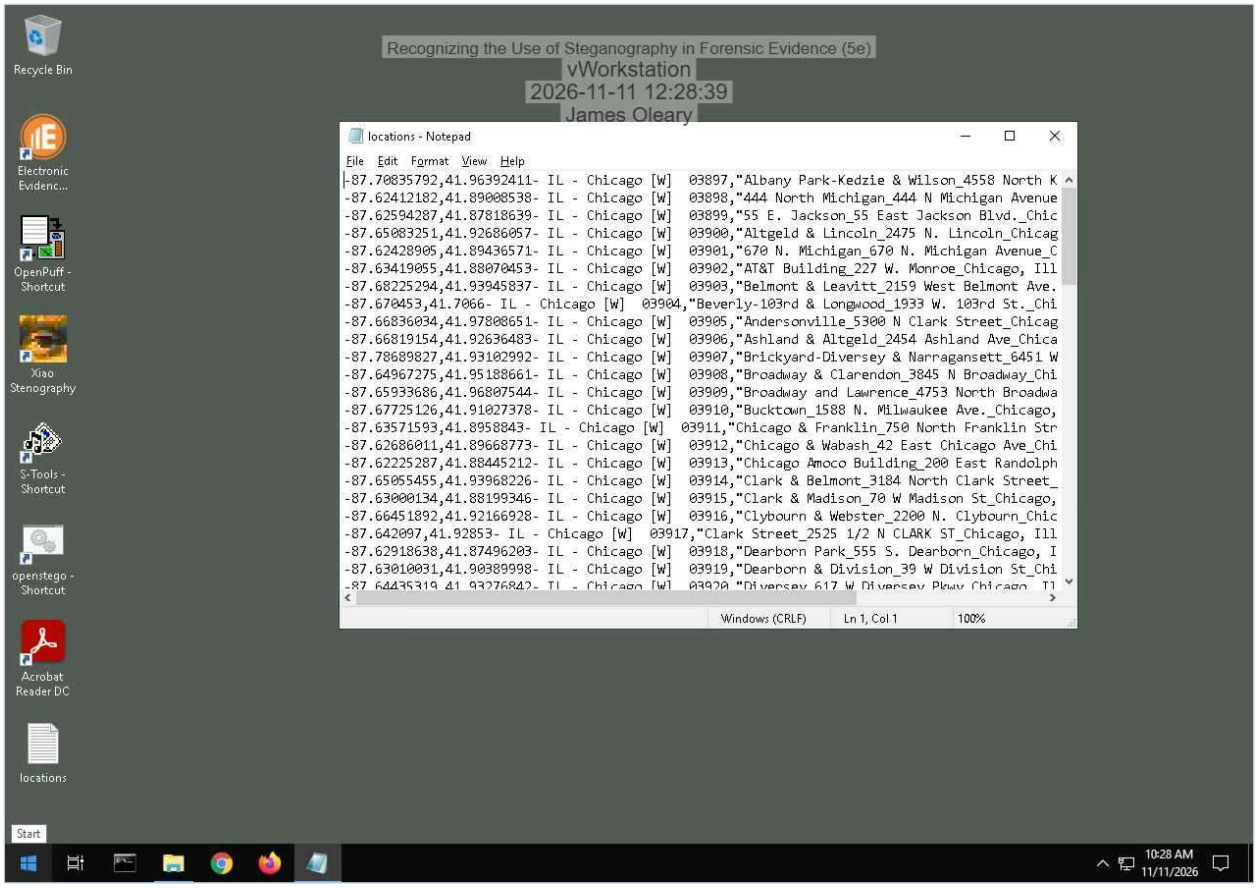


Figure 4. *Part 3, Required Screenshot 4 (Step 20): Contents of the file extracted by OpenPuff*

5.3 Findings

Finding	Result
Finding 1: ReadMe passphrase (Step 4)	Lakeview
Finding 2: Hidden file contents (Step 21)	GPS Coordinates and Addresses located in the Chicago area.
Finding 2: Relevance to the investigation (Step 21)	Choosing to hide the locations list with steganography, combined with the renamed copy of OpenPuff, points to a deliberate effort to keep these locations from being found. The next steps are to determine how these locations connect to the case, identify the sender of “lakeview.bmp”, and then scan the remaining images for additional hidden data.

6. Research Question: Key Lessons in Steganography and Hidden-Data Analysis

The first lesson that I learned was that file names prove nothing. OpenPuff was stored as temp17.exe.exe but the MD5 hash matched a tool in the steganography group that is a part of the E3 hash database. Kent et al. (2006) warn that file extensions are easy to change and recommend identifying files by content, including comparing hashes against sets of known files.

Another lesson I learned was that hidden data doesn't have to impact an image. In paint, lakeview.bmp was a normal looking lake photo, yet StegExpose still estimated it had 31289 bytes of hidden data. LSB embedding changes each pixel value by a tiny bit, which the eye cannot pick up but shifts the images stats (Fridrich, 2009).

The final lesson I learned was that finding hidden data is not the same as reading it. I could only recover the locations file because a tool was known and a passphrase was found in a simple Readme. MITRE (n.d.) describes steganography as a method to keep hidden information from being detected. The tool and key are usually what an analyst is missing. Extracting from the exported copy also left the original evidence intact which follows NIST guidance to examine copies and preserve integrity (Kent et al., 2006).

7. Conclusion

Each part of the lab depended on a discovery that was made prior. The hash match in E3 showed OpenPuff was on the drive as temp17.exe.exe, StegExpose pointed at lakeview.bmp as the carrier, and then the discovered Readme passphrase let OpenPuff recover the hidden list of Chicago addresses and GPS coordinates.

References

Boehm, B. (2014). *StegExpose: A tool for detecting LSB steganography* (arXiv:1410.6656). arXiv. https://doi.org/10.48550/arXiv.1410.6656

Fridrich, J. (2009). *Steganography in digital media: Principles, algorithms, and applications*. Cambridge University Press.

Kent, K., Chevalier, S., Grance, T., & Dang, H. (2006). *Guide to integrating forensic techniques into incident response* (NIST Special Publication 800-86). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-86

MITRE. (n.d.). *Obfuscated files or information: Steganography (T1027.003)*. MITRE ATT&CK. Retrieved November 11, 2026, from https://attack.mitre.org/techniques/T1027/003/

Appendix A: MITRE ATT&CK Mapping

Technique	Name	Observed in Lab
T1027.003	Obfuscated Files or Information: Steganography	Data concealed within an image file (Parts 2–3)
T1001.002	Data Obfuscation: Steganography	Hidden content embedded in received media
T1036	Masquerading	Steganography program renamed on the drive (Part 1)